COM6115: Text Processing

Student Name:- Bhagawan Sekhara Raju
Course:- COM6115: Text Processing
Professor Name:- Varvara Papazoglou & Dr Carolina Scarton
Submission Date:- 12/12/2024

COM6115 Text Processing

Assessment: Sentiment Analysis

STEP-2:

Run Naïve Bayes on Rotten Tomatoes Data

1.) There are two files positive-words.txt and negative-words.txt in which python uses a build in function which is open function which is used to iterate the file line by line or reading the entire content. r indicates read mode where no changes in the file contents and it has encoding="ISO-8859-1 it contains character outside the ASCII Range. These files contain metadata and it is commented by prefix by; which should not be included in the word list.strip() function is used to clean words without trailing spaces and if word and not word.startswith(",") is used to filter unwanted lines from the text before adding remaining words to the list.

```python
posSentencesNokia = open('nokia-pos.txt', 'r')
posSentencesNokia = re.split(r'\n', posSentencesNokia.read())

negSentencesNokia = open('nokia-neg.txt', 'r', encoding="ISO-8859-1")
negSentencesNokia = re.split(r'\n', negSentencesNokia.read())

posDictionary = open('positive-words.txt', 'r', encoding="ISO-8859-1")
posWordList = [] # edit this to get the list of the positive sentiment words in the dictionary
for word in posDictionary:
    word=word.strip()
    if word and not word.startswith(","):
      posWordList.append(word)


negDictionary = open('negative-words.txt', 'r', encoding="ISO-8859-1")
negWordList = [] # edit this to get the list of the negative sentiment words in the dictionary
for word in negDictionary:
    word=word.strip()
    if word and not word.startswith(","):
     negWordList.append(word)

for i in posWordList:
    sentimentDictionary[i] = 1
for i in negWordList:
    sentimentDictionary[i] = -1

# Create Training and Test Datsets:
# We want to test on sentences we haven't trained on,
# to see how well the model generalses to previously unseen sentences
```

```python
def cal_accurracy(correct, total):
    return correct / total if total !=0  else 0

def cal_precision(correct_pos, total_pos_pred):
    return correct_pos / total_pos_pred if total_pos_pred !=0 else 0

def calculate_recall(correct_pos, total_pos):
    return correct_pos / total_pos if total_pos != 0 else 0

def calculate_f1_score(precision_pos, recall_pos):
    return (2 * precision_pos * recall_pos) / (precision_pos + recall_pos) if (precision_pos + recall_pos) > 0 else 0
def testBayes(sentencesTest, dataName, pWordPos, pWordNeg, pWord,pPos):

    print("Naive Bayes classification")
    pNeg=1-pPos

    # These variables will store results
    total=0
    correct=0
    totalpos=0
    totalpospred=0
    totalneg=0
    totalnegpred=0
    correctpos=0
    correctneg=0
```

```python
                correct+=1
                correctneg+=1
                totalnegpred+=1
            else:
                correct+=0
                totalpospred+=1
                if PRINT_ERRORS:
                    print ("ERROR (neg classed as pos %0.2f):" %prob + sentence)
    accuracy = cal_accurracy(correct, total)
    accuracy = round(accuracy * 100) / 100
    precisionPos = cal_precision(correctpos, totalpospred)
    precisionPos = round(precisionPos * 100) / 100
    recallPos = calculate_recall(correctpos, totalpos)
    recallPos = round(recallPos * 100) / 100
    f1Pos = calculate_f1_score(precisionPos, recallPos)
    recallPos = round(recallPos * 100) / 100
    f1Pos = round(f1Pos * 100) / 100

    precisionNeg = cal_precision(correctneg, totalnegpred)
    recallNeg = calculate_recall(correctneg, totalneg)
    f1Neg = calculate_f1_score(precisionNeg, recallNeg)

    # Print results
    print(f"Accuracy: {accuracy:.4f} ")
    print(f"Precision (Positive): {precisionPos:.4f}, Recall (Positive): {recallPos:.4f}, F1-Score (Positive): {f1Pos:.4f}")
    print(f"Precision (Negative): {precisionNeg:.4f}, Recall (Negative): {recallNeg:.4f}, F1-Score (Negative): {f1Neg:.4f}")
```

```
I NAIVE BAYES CLASSIFIER

1.Films train Dataset
Naive Bayes classification
Accuracy: 0.8900
Precision (Positive): 0.9000, Recall (Positive): 0.8800, F1-Score (Positive): 0.8900
Precision (Negative): 0.8863, Recall (Negative): 0.9013, F1-Score (Negative): 0.8938

2.Films test Dataset
Naive Bayes classification
Accuracy: 0.7600
Precision (Positive): 0.7700, Recall (Positive): 0.7700, F1-Score (Positive): 0.7700
Precision (Negative): 0.7480, Recall (Negative): 0.7495, F1-Score (Negative): 0.7488

3.Nokia Dataset
Naive Bayes classification
Accuracy: 0.6100
Precision (Positive): 0.7900, Recall (Positive): 0.5900, F1-Score (Positive): 0.6800
Precision (Negative): 0.4016, Recall (Negative): 0.6375, F1-Score (Negative): 0.4928
```

In film reviews dataset, this model has an accuracy of 89% and precision of 90% and recall of 88% which illustrates that the model has performed well on this dataset.

But in testing the model on unseen test dataset the accuracy has dropped below 80% along with precision and recall which is 77% and 77% for both as we can come to conclusion that the model overfits in training data and underperforms on test data.

In Naïve Bayes model underachieves on the Nokia dataset with accuracy of 61% but precision an has quite high on the positive class of 79% whereas in negative class it drops to half of positive class which is 40%. As this shows that this model is not able to rectify negative sentiment in Nokia Reviews dataset.

STEP-3:

Run Naïve Bayes on Nokia Data

Now let's look at Naive Bayes classification on training data on Nokia product reviews.

1.)

```
# run naive bayes classifier on datasets
print("\nI NAIVE BAYES CLASSIFIER")
print("\n1.Films train Dataset")
testBayes(sentencesTrain,  "Films (Train Data, Naive Bayes)\t", pWordPos, pWordNeg, pWord,0.5)
print("\n2.Films test Dataset")
testBayes(sentencesTest,  "Films  (Test Data, Naive Bayes)\t", pWordPos, pWordNeg, pWord,0.5)
print("\n3.Nokia Dataset")
testBayes(sentencesNokia, "Nokia   (All Data,  Naive Bayes)\t", pWordPos, pWordNeg, pWord,0.7)
```

```
I NAIVE BAYES CLASSIFIER

3.Nokia Dataset
Naive Bayes classification
Accuracy: 0.6100
Precision (Positive): 0.7900, Recall (Positive): 0.6000, F1-Score (Positive): 0.6800
Precision (Negative): 0.4032, Recall (Negative): 0.6250, F1-Score (Negative): 0.4902
```

2)

While observing the result on the Nokia dataset, we can conclude that model is not a good fit to solve the issues in the Nokia dataset with the accuracy of 61% we can conclude that model is average in predicting sentiments.

For positive dataset the model is strongly accurate in predicting as it has a precision of 79% and recall nearly misses out one fourth of the positive reviews and f1 score of 68% thus balancing out the performance positive class.

In Negative dataset the precision is 40% which is not appropriate and in recall it is 62% which is quite better than positive response and lastly in F1 score is less than 50% which is not good in identifying negative sentiments.

STEP – 4:

1) What is being learnt by the model?

```
NEGATIVE:
['unfunny', 'routine', 'generic', 'badly', 'mediocre', 'lame', 'mindless', 'stale', 'shoot', 'pointless', 'poorly
'tiresome', 'dull', 'apparently', 'harvard', 'plodding', 'stupid', 'disaster', 'tired', 'meandering', 'horrible',
'pinocchio', 'junk', 'stealing', 'animal', 'amateurish', 'banal', 'trite', 'ballistic', 'seagal', 'offensive', 'u
'incoherent', 'crap', 'leaden', 'kung', 'product', 'lifeless', 'inane', 'soggy', 'unnecessary', '51', 'tv', 'half
'missed', 'pile', 'crass', 'comparison', 'named', 'choppy', 'clumsily', 'listless', 'flat', 'numbers', 'britney',
'wannabe', 'store', 'jokes', 'imitation', 'unfocused', 'settles', 'arts', 'relentlessly', 'scattered', 'writers'

POSITIVE:
['superbly', 'conduct', 'leigh', 'gently', 'poem', 'finely', 'para', 'kissinger', 'portrayal', 'tradition', 'sple
'grandeur', 'lane', 'undeniably', 'absorbing', 'lovers', 'holocaust', 'honesty', 'droll', 'heartbreaking', 'unfli
'pianist', 'breathtaking', 'sobering', 'ingenious', 'delightfully', 'literary', 'smarter', 'deft', 'joyous', 'por
'warm', 'evocative', 'playful', 'startling', 'iranian', 'resonant', 'wrenching', 'potent', 'sides', 'grown', 'tra
'challenging', 'gradually', 'record', 'subversive', 'answers', 'capture', 'sadness', 'flawed', 'powerful', 'marth
'heartwarming', 'captivating', 'detailed', 'unexpected', 'spare', 'dazzling', 'document', 'provides', 'captures',
'chilling', 'pulls', 'gem', 'lively', 'wry', 'quietly', 'wonderfully', 'intense', 'refreshing', 'wonderful', 'aff
```

2) Here we can observer that

- Some words like Harvard, leigh, Chan, Sara, Car, or Kissinger are some neutrals words found negative dictionary.
- Positive words- In positive words there are words like beautiful, touching, beauty and joyous are the words associates with positive sentiments and “Jealousy, “Desperation”, “disturbing”, “Heartbreaking”, etc are the negative words found in positive words.
- In Negative words – “waste, unfunny, badly, boring, poorly etc which aligns to human emotions.
- As we have used random function to find the distribution of the sentiment terms and now let count the changes after each cycle

Number of Iteration	Positive word count	Negative word count	Total
1)	43	42	85
2)	50	42	92
3)	46	43	89
4)	53	41	94
5)	49	46	96

MEAN	91

The function to count sentiment

```
pos_words =0
neg_words =0
for word in sentimentDictionary:
  if word in tail:
    pos_words+=1
  if word in head:
    neg_words+=1

print("\nNumber of positive words:", pos_words)
print("\nNumber of negative words:", neg_words)
```

STEP – 5:

How does a rule-based system compare to Naïve Bayes?

```python
def cal_accurracy(correct, total):
    return correct / total if total !=0  else 0

def cal_precision(correct_pos, total_pos_pred):
    return correct_pos / total_pos_pred if total_pos_pred !=0 else 0

def calculate_recall(correct_pos, total_pos):
    return correct_pos / total_pos if total_pos 0 else 0

def calculate_f1_score(precision_pos, recall_pos):
    return (2 * precision_pos * recall_pos) / (precision_pos + recall_pos) if (precision_pos + recall_pos) > 0 else 0

    accuracy = cal_accurracy(correct, total)
    accuracy = round(accuracy * 100) / 100
    precisionPos = cal_precision(correctpos, totalpospred)
    precisionPos = round(precisionPos * 100) / 100
    recallPos = calculate_recall(correctpos, totalpos)
    recallPos = round(recallPos * 100) / 100
    f1Pos = calculate_f1_score(precisionPos, recallPos)
    recallPos = round(recallPos * 100) / 100
    f1Pos = round(f1Pos * 100) / 100

    precisionNeg = cal_precision(correctneg, totalnegpred)
    recallNeg = calculate_recall(correctneg, totalneg)
    f1Neg = calculate_f1_score(precisionNeg, recallNeg)
    print(f"Accuracy: {accuracy:.4f} ")
    print(f"Precision (Positive): {precisionPos:.4f}, Recall (Positive): {recallPos:.4f}, F1-Score (Positive): {f1Pos:.4f}")
    print(f"Precision (Negative): {precisionNeg:.4f}, Recall (Negative): {recallNeg:.4f}, F1-Score (Negative): {f1Neg:.4f}")
```

```
II RULE BASED CLASSIFIER

1.Films Dataset
Dictionary-based classification
Accuracy: 0.6600
Precision (Positive): 0.6800, Recall (Positive): 0.5800, F1-Score (Positive): 0.6300
Precision (Negative): 0.6338, Recall (Negative): 0.7350, F1-Score (Negative): 0.6807

2.Films Dataset
Dictionary-based classification
Accuracy: 0.6300
Precision (Positive): 0.6600, Recall (Positive): 0.5200, F1-Score (Positive): 0.5800
Precision (Negative): 0.6053, Recall (Negative): 0.7285, F1-Score (Negative): 0.6612

3.Nokia Dataset
Dictionary-based classification
Accuracy: 0.8000
Precision (Positive): 0.8800, Recall (Positive): 0.8200, F1-Score (Positive): 0.8500
Precision (Negative): 0.6383, Recall (Negative): 0.7500, F1-Score (Negative): 0.6897
```

a) How does the dictionary- based approach compare to Naïve bayes on the two domains?

In both approach there are differences in results, In Navies bayes basically outperforms dictionary based where is has performed well on the Films dataset but on the other hand it's the opposite when we consider dictionary based where it has performed well on the Nokia dataset.

b) What conclusions do you draw about statistical and rule-based approaches from your observations?

Navies bayes: In this approach it adapts varies domains and different types of text as it learns data by itself. It performs well on large dataset as it recognizes the patterns in certain contexts.as it is good in understanding the relationship in words and there contexts.so this model with corresponding target value performs well but fails to performs on the Nokia dataset.

Rule based: In this approach it works well with clear sentiment words but only struggles with those which are specified directly and does not consider context of the words so that's the reason to failed to give a good performance on the film dataset.

c) Write a function to improve rule-based system e.g., to take into account negation, diminisher rules, etc and justify your decisions. Run the program again and analyse the results on both datasets

```
def modified_testDictionary(sentencesTest, dataName, sentimentDictionary, threshold):

    print("Dictionary-based classification")
    total = 0
    correct = 0
    totalpos = 0
    totalneg = 0
    totalpospred = 0
    totalnegpred = 0
    correctpos = 0
    correctneg = 0

    intensifiers = {
        "totally", "utterly", "absolutely", "incredibly", "highly", "too", "very",
        "really", "quite", "significantly", "fairly", "particularly", "super",
        "notably", "super", "crazy", "massively"
    }
    diminishers = {
        "hardly", "kinda", "marginally", "sort of", "a bit", "relatively", "a little",
        "barely", "moderately", "somewhat", "faintly", "almost", "rather",
        "negligible", "to a lesser context", "might", "could", "minimal", "in part",
        "seldom", "not much"
    }
    negations = {
        "neither", "no", "nor", "not", "none", "no one", "nowhere", "nobody",
        "wasn't", "hasn't", "haven't", "hadn't", "doesn't", "isn't", "aren't",
        "weren't", "can't", "cannot", "barely", "rarely", "scarcely"
    }
```

```python
for sentence, sentiment in sentencesTest.items():
    Words = re.findall(r"[\w']+", sentence)
    score = 0
    negate = False
    dim = False
    inten = False

    for word in Words:
        if word in negations:
            negate = True
            continue
        if word in diminishers:
            dim = True
            continue
        if word in intensifiers:
            inten = True
            continue

        if word in sentimentDictionary:
            word_score = sentimentDictionary[word]


            if negate:
                word_score *= -1
                negate = False


            if word in intensifiers:
                word_score *= 2
```

```
            if word in intensifiers:
                word_score *= 2


            if word in diminishers:
                word_score *= 0.5


            score += word_score


    total += 1
    if sentiment == "positive":
        totalpos += 1
        if score >= threshold:
            correct += 1
            correctpos += 1
            totalpospred += 1
        else:
            totalnegpred += 1
    else:
        totalneg += 1
        if score < threshold:
            correct += 1
            correctneg += 1
            totalnegpred += 1
        else:
```

```
            totalpospred += 1
        else:
            totalnegpred += 1
    else:
        totalneg += 1
        if score < threshold:
            correct += 1
            correctneg += 1
            totalnegpred += 1
        else:
            totalpospred += 1

accuracy = cal_accurracy(correct, total)
accuracy = round(accuracy * 100) / 100
precisionPos = cal_precision(correctpos, totalpospred)
precisionPos = round(precisionPos * 100) / 100
recallPos = calculate_recall(correctpos, totalpos)
recallPos = round(recallPos * 100) / 100
f1Pos = calculate_f1_score(precisionPos, recallPos)
recallPos = round(recallPos * 100) / 100
f1Pos = round(f1Pos * 100) / 100

precisionNeg = cal_precision(correctneg, totalnegpred)
recallNeg = calculate_recall(correctneg, totalneg)
f1Neg = calculate_f1_score(precisionNeg, recallNeg)


print(f"Accuracy: {accuracy:.4f}")
```

Here in this function, we have taken a list of Negations, Diminisher and Intensifier for helping to do better analysis on the dataset and I have used for, if else statements and Boolean flags to get the scores accordingly which help in increasing the accuracy of the Nokia dataset.

New approach

```
II RULE BASED CLASSIFIER

1.Films Dataset
Dictionary-based classification
Accuracy: 0.6500
Precision (Positive): 0.6800, Recall (Positive): 0.5500, F1-Score (Positive): 0.6100
Precision (Negative): 0.6232, Recall (Negative): 0.7442, F1-Score (Negative): 0.6783

2.Films Dataset
Dictionary-based classification
Accuracy: 0.6600
Precision (Positive): 0.6800, Recall (Positive): 0.5700, F1-Score (Positive): 0.6200
Precision (Negative): 0.6453, Recall (Negative): 0.7403, F1-Score (Negative): 0.6895

3.Nokia Dataset
Dictionary-based classification
Accuracy: 0.8200
Precision (Positive): 0.9100, Recall (Positive): 0.8200, F1-Score (Positive): 0.8600
Precision (Negative): 0.6566, Recall (Negative): 0.8125, F1-Score (Negative): 0.7263
```

STEP – 6:

Comment out all but one of the testBayes and testDictionary calls (one for each,
two in total).

2. At the top of the program, set PRINT_ERRORS=1

```
I NAIVE BAYES CLASSIFIER

1.Films train Dataset
Naive Bayes classification
ERROR (pos classed as neg 0.15):the rock is destined to be the 21st century's new " conan " and that he's going to make a splash even greater than arnold schwarzenegger , jean-cl
ERROR (pos classed as neg 0.23):effective but too-tepid biopic
ERROR (pos classed as neg 0.35):if you sometimes like to go to the movies to have fun , wasabi is a good place to start .
ERROR (pos classed as neg 0.04):perhaps no picture ever made has more literally showed that the road to hell is paved with good intentions .
ERROR (pos classed as neg 0.11):ultimately , it ponders the reasons we need stories so much .
ERROR (pos classed as neg 0.25):if there's a way to effectively teach kids about the dangers of drugs , i think it's in projects like the ( unfortunately r-rated ) paid .
ERROR (pos classed as neg 0.45):manages to be original , even though it rips off many of its ideas .
ERROR (pos classed as neg 0.29):if this movie were a book , it would be a page-turner , you can't wait to see what happens next .
ERROR (pos classed as neg 0.35):a sports movie with action that's exciting on the field and a story you care about off it .
ERROR (pos classed as neg 0.23):i enjoyed time of favor while i was watching it , but i was surprised at how quickly it faded from my memory .
ERROR (pos classed as neg 0.14):somewhere short of tremors on the modern b-scene : neither as funny nor as clever , though an agreeably unpretentious way to spend ninety minutes
ERROR (pos classed as neg 0.28): " extreme ops " exceeds expectations . good fun , good action , good acting , good dialogue , good pace , good cinematography .
ERROR (pos classed as neg 0.30):it's not exactly a gourmet meal but the fare is fair , even coming from the drive-thru .
ERROR (pos classed as neg 0.29):as lo-fi as the special effects are , the folks who cobbled nemesis together indulge the force of humanity over hardware in a way that george luca
ERROR (pos classed as neg 0.38):more a load of enjoyable , conan-esque claptrap than the punishing , special-effects soul assaults the mummy pictures represent .
ERROR (pos classed as neg 0.03):maybe it's just because this past year has seen the release of some of the worst film comedies in decades . . . but honestly , analyze that really
ERROR (pos classed as neg 0.43):the kind of nervous film that will either give you a mild headache or exhilarate you .
ERROR (pos classed as neg 0.21):watching beanie and his gang put together his slasher video from spare parts and borrowed materials is as much fun as it must have been for them t
ERROR (pos classed as neg 0.36):children may not understand everything that happens -- i'm not sure even miyazaki himself does -- but they will almost certainly be fascinated , a
ERROR (pos classed as neg 0.18):my goodness , queen latifah has a lot to offer and she seemed to have no problem flaunting her natural gifts . she must have a very strong back .
ERROR (pos classed as neg 0.20):though this film can be clumsy , its ambitions are equally -- and admirably -- uncommercial .
ERROR (pos classed as neg 0.21):day is not a great bond movie , but it is a good bond movie , which still makes it much better than your typical bond knock-offs .
ERROR (pos classed as neg 0.36):you wouldn't call the good girl a date movie ( an anti-date movie is more like it ) , but when it's good , it's good and horrid .
ERROR (pos classed as neg 0.16):it would be interesting to hear from the other side , but in talk to her , the women are down for the count .
ERROR (pos classed as neg 0.13):the new guy does have a heart . now , if it only had a brain .
ERROR (pos classed as neg 0.30):the film's greatest asset is how much it's not just another connect-the-dots , spy-on-the-run picture .
ERROR (pos classed as neg 0.45):i don't know precisely what to make of steven soderbergh's full frontal , though that didn't stop me from enjoying much of it .
ERROR (pos classed as neg 0.41):you could say that it's slow at times , you could say that a few of the characters act in ways that real people wouldn't , but one thing you could
ERROR (pos classed as neg 0.27):asia authors herself as anna battista , an italian superstar and aspiring directress who just happens to be her own worst enemy .
ERROR (pos classed as neg 0.42):spielberg is the rare director who does not want to invite viewers to gawk at or applaud his special effects . he just wants them to be part of th
ERROR (pos classed as neg 0.30):girls gone wild and gone civil again
ERROR (pos classed as neg 0.40):this is such a high-energy movie where the drumming and the marching are so excellent , who cares if the story's a little weak .
```

Explanation of why the model is making mistakes

- After analysing the errors, we found out that in testDictionary it struggles with all of the 3 modifiers where the accuracy is decreased for instance I can't it was amazing which was

supposed to be negative but predict it has positive so the model is unable to predict properly.

- Some words like a bit or hardly which declines the sentiments as the model will overlook these words eventually.

- In TestBayes the model lacks the understanding of word order which in turn leads to errors in negations and modifiers. The model will overlook for most repeated words which leads to misinterpretation for example “not good” where it misinterprets good as it is frequently repeated and classifying them towards positive when the actual sentiment is neutral or negative.

- Rule based (Dictionary)- The threshold value is so poor that it is overly positive or negative

- In negations, Diminishers and intensifier are considered as, complex sentence which confuses the model and which impacts the performance as well.